The results look the same as with the process pool, except they were in a random order. That's really all there is to distributing a function across multiple CPU processors.

5.3. Advantage actor-critic

Now that we know how to distribute computation across processes, we can get back to the real reinforcement learning. In this section we'll put together the pieces of the full distributed advantage actor-critic model. To allow fast training and to compare the results to the previous chapter, we will again use the CartPole game as our test environment. If you choose, though, you can easily adapt the algorithm to a more difficult game such as Pong in OpenAI Gym; you can find such an implementation on this chapter's GitHub page: http://mng.bz/JzKp.

So far we've presented the actor and critic as two separate functions, but we can combine them into a single neural network with two output "heads." That's what we'll do in the following code. Instead of a normal neural network that returns a single vector, it can return two different vectors: one for the policy and one for the value. This allows for some parameter sharing between the policy and value that can make things more efficient, since some of the information needed to compute values is also useful for predicting the best action for the policy. But if a two-headed neural network seems too exotic right now, you can go ahead and write two separate neural networks—it will work just fine. Let's look at some pseudocode for the algorithm. Then we'll translate it to Python.

Listing 5.3. Pseudocode for online advantage actor-critic

```
gamma = 0.9
for i in epochs:                                                    1
    state = environment.get_state()                                 2
    value = critic(state)                                           3
    policy = actor(state)                                           4
    action = policy.sample()                                        5
    next_state, reward = environment.take_action(action)
    value_next = critic(next_state)                                 6
    advantage = reward + (gamma * value_next – value)               7
    loss = -1 * policy.logprob(action) * advantage                  8
    minimize(loss)
```